



AccoTEST STS8300 应用培训

QTMUe 单板硬件及应用

2022-1-11

Make Testing More Valuable

目录

- 单板特性
- 指标
- 原理框图及外观结构
- 槽位及对应PogoPin定义
- 原理图设计、PCB布局注意事项
- 函数样例
 - SetInSource
 - SetMatrix
 - Arm外部触发

• 单板特性

SM8211 四通道时间测量模块是 STS 8300 系统测试头内的标准模块，模块提供四通道时间测量功能，其主要特性如下：

- ❑ 系统最多支持 4 块 QTMUe 模块工作
- ❑ 每模块有 4 个时间测量单元，每单元有 2 个输入接口
- ❑ 3 个时间量程档
 - 5ns~950ns, 800ns~4ms, 5ns~40s
- ❑ 五个电压量程档
 - $\pm 50V$, $\pm 25V$, $\pm 10V$, $\pm 5V$, $-2\sim+6V$
- ❑ 最高分辨率<30ps
- ❑ 测量频率 $\leq 50MHz$
- ❑ 每个时间测量单元有 64K 测量结果存储深度
- ❑ 支持外部触发功能
- ❑ 支持多工位并行测试工作模式

• 指标

Measurement Unit				
Number	4 bank/board			
Measurement	time		frequency	
Capability	5ns~40s		0.1Hz~50MHz	
Measurement Type	Propagation delay, rise/fall time, pulse width, frequency, duty cycle, event, etc.			
Range	5ns~950ns	800ns~4ms	5ns~40s	
Accuracy	$\pm(2\text{ns}+0.1\%\text{Rdg})$ (1)	$\pm(2\text{ns}+0.1\%\text{Rdg})$ (1)	5ns~5us	5us~40s
			$\pm(10\text{ns}+0.1\%\text{Rdg})$ (1)	$\pm(5\text{ns}+0.1\%\text{Rdg})$ (1)
Resolution	<50ps	<30ps	5ns	
Sample Memory	64K/channel			

(1) For signals with >200V/us slew rate

• 指标

Input Channel		
Number	8/board	
	Analog Mode	Digital Mode
Input Voltage Range	$\pm 50V / \pm 25V / \pm 10V / \pm 5V$	-2~+6V
Input Bandwidth	>50MHz	>200MHz
Input Impedance	1M Ω	>10M Ω
LPF	Direct Pass/10MHz/1MHz/100KHz	Direct Pass
Trigger Level Range	$\pm 50V / \pm 25V / \pm 10V / \pm 5V$	-2~+6V
Trigger Level Resolution	14bit	

注意：

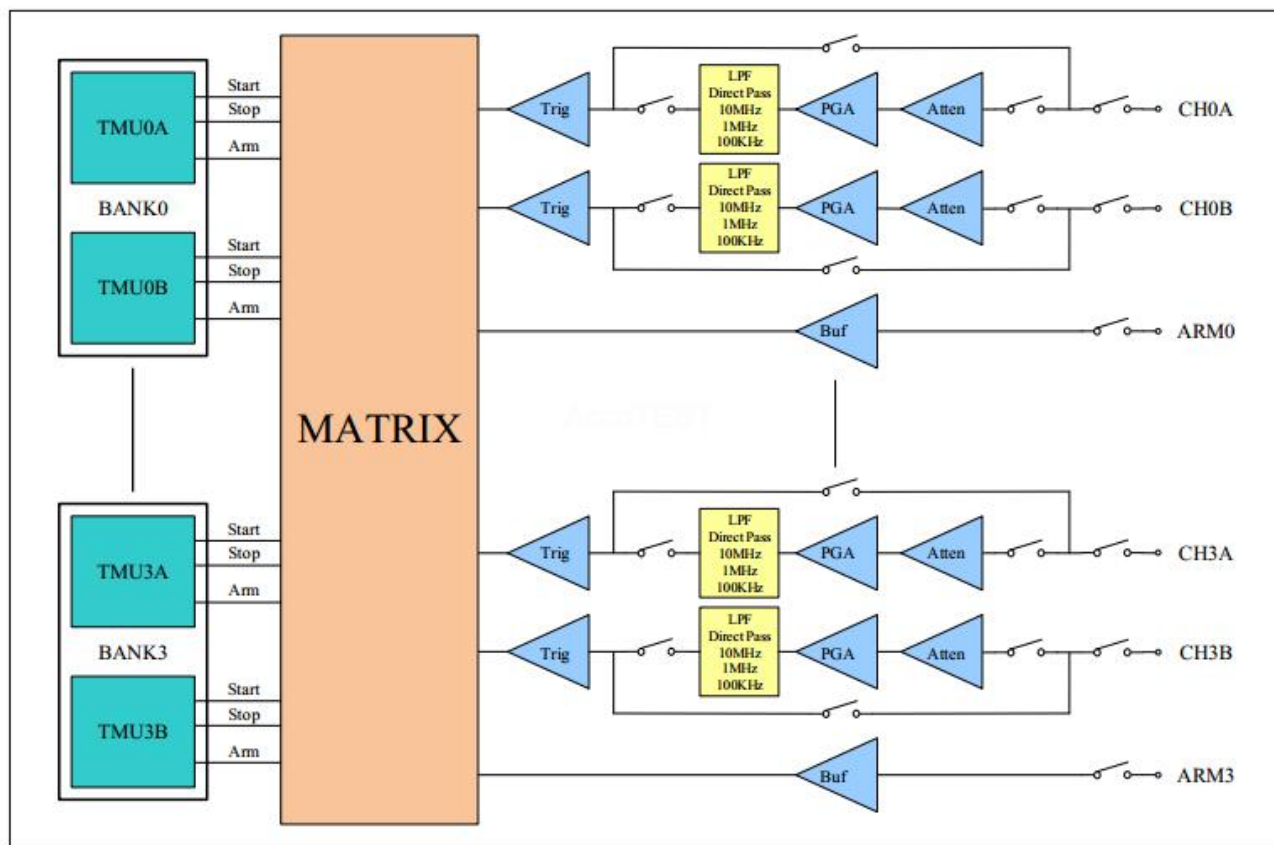
Digital Mode 的电压范围极限为-7~+11V，超出会对 QTMUe 造成永久性损坏，建议在标称电压范围-2~+6V 内使用。

- 指标

ARM(External Trigger)

Number	4/board
Input Voltage Range	0~5.5V
VIH	2.0V-5.5V
VIL	0V-0.8V
Input Bandwidth	>200MHz

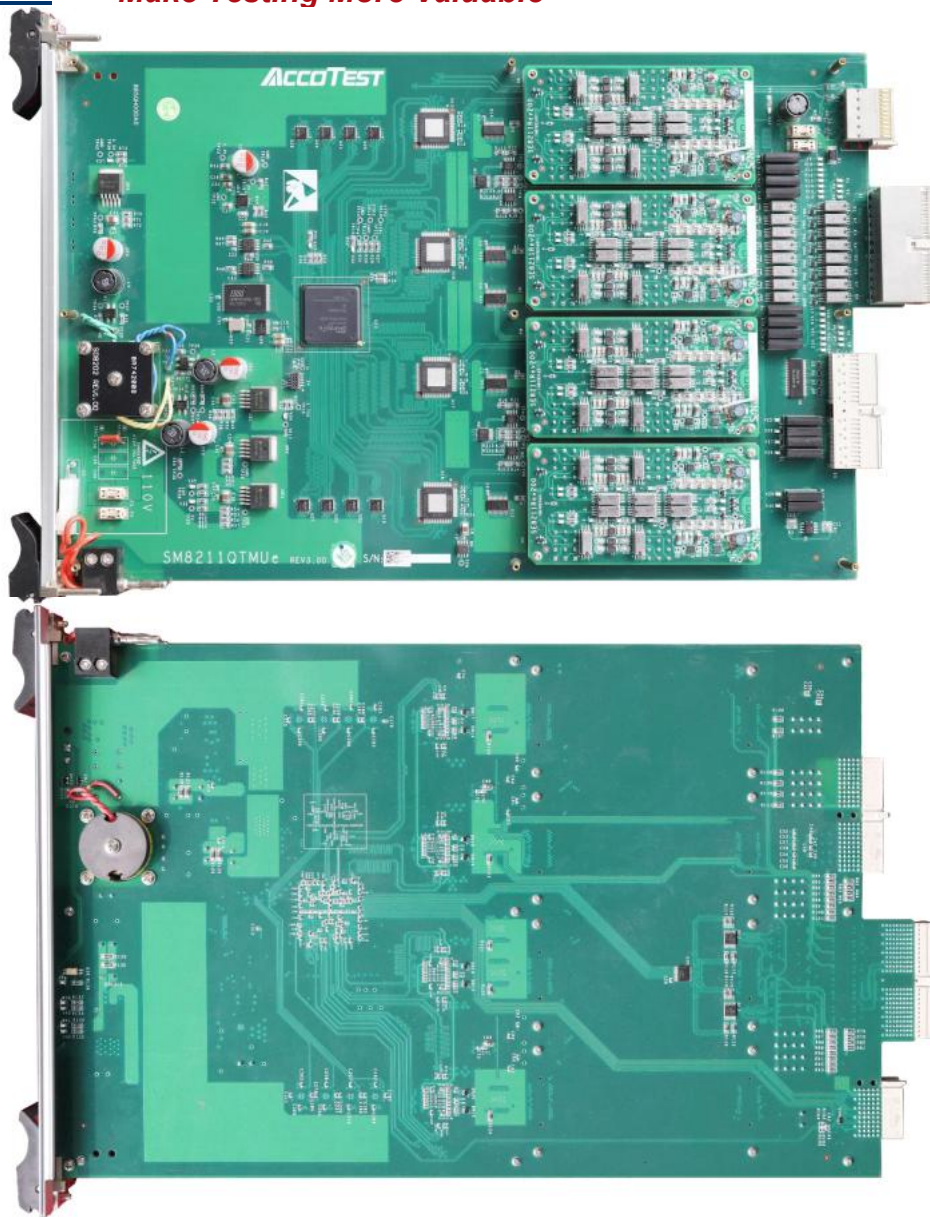
原理框图



QTMUe Block Diagram

其中 CH0A~CH3B 引入待测时间信号， ARM0~ARM3 引入外部触发信号

- 外观结构



槽位及对应PogoPin定义

QTMUe对应的槽位（TIB类型相关）

33	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	34
NULL	Analog	Analog	Analog	Analog	Analog	Analog	Analog	Analog	DCM	QTMUe	Analog	Analog	Analog	Analog	Analog	Analog	CBite
	Analog	Analog	Analog	Analog	Analog	Analog	Analog	Analog	DCM	QTMUe	Analog	Analog	Analog	Analog	Analog	Analog	Cal
36	32	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	35
33	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	34
NULL	Analog	Analog	Analog	Analog	Analog	Analog	TMU	Digital	Digital	TMU	Analog	Analog	Analog	Analog	Analog	Analog	CBite
	Analog	Analog	Analog	Analog	Analog	Analog	TMU	Digital	Digital	TMU	Analog	Analog	Analog	Analog	Analog	Analog	CAL
36	32	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	35

A28D4 Config

A24D8 Config

槽位及对应PogoPin定义

From PogoPin Tool

P107_slot10slot9

	I	H	G	F	E	D	C	B	A
	DGND	ESYNC1	DGND	DUTG1	DGND	ESYNC0	DGND		
1	P_34	DGND	P_24	DGND	P_14	DGND	P_4	DGND	DUTG0
2	DGND	P_29	DGND	P_19	DGND	P_9	DGND	P_0	DGND
3	P_35	DGND	P_25	DGND	P_15	DGND	P_5	DGND	CHA_0
4	DGND	P_30	DGND	P_20	DGND	P_10	DGND	P_1	DGND
5	P_36	DGND	P_26	DGND	P_16	DGND	P_6	DGND	CHA_1
6	DGND	P_31	DGND	P_21	DGND	P_11	DGND	P_2	DGND
7	P_37	DGND	P_27	DGND	P_17	DGND	P_7	DGND	CHA_2
8	DGND	P_32	DGND	P_22	DGND	P_12	DGND	P_3	DGND
9	P_38	DGND	P_28	DGND	P_18	DGND	P_8	DGND	CHA_3
10	DGND	P_33	DGND	P_23	DGND	P_13	DGND	PCASIO	DGND
11	PCASO2	DGND	PCASIO1	DGND	PCASO1	DGND	PCASO0		
12									

P108_slot34slot10slot9slot36

	K	J	I	H	G
			SCL		
1		DGND	SDA	CBITe_32	CBITe_33
2	DGND	ARM_0		CBITe_40	CBITe_41
3	CHB_0	DGND	JGND	CBITe_48	CBITe_49
4	DGND	ARM_1	JGND	CBITe_56	CBITe_57
5	CHB_1	DGND			
6	DGND	ARM_2			
7	CHB_2	DGND			
8	DGND	ARM_3			
9	CHB_3	DGND	GUARD		
10	DGND		SENSE		
11			FORCE	J+5V	J-5V
12					J+

QTMUe

Slot:

slot10

Serial NO:

0

☐ AutoSwitch

Config File Path:

C:/AccoTest/AccoTEST System/ConfigEx.slot

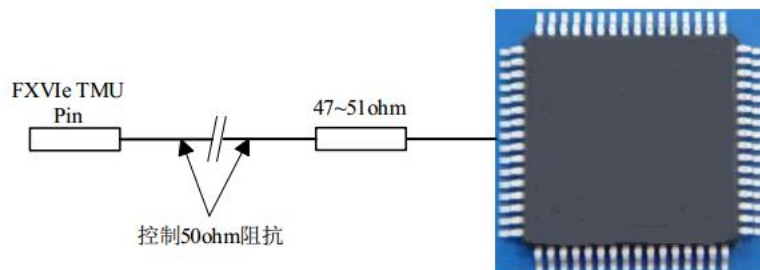
Cell Channel And Slot:

P122 P121 P120 P119 P118 P117 P116 P115 P114
P123 P124 P125 P126 P127 P128 P129 P130
P101 P102 P103 P104 P105 P106 P107 P108 P109 P110

• 原理图设计、PCB布局及注意事项

1. 阻抗匹配

QTMUe 所有输入信号在layout时都需考虑50ohm 阻抗匹配，另外，建议在器件输出管脚近端串接一个47~51ohm 电阻。



2. 测量单元 /物理通道

一块QTMUe板卡包含四个测量单元/物理通道，每个测量单元包含两个输入通道CHA 和CHB，CHA和CHB 相互独立，可作为两个TMU 使用。

3. 输入通道/用户通道

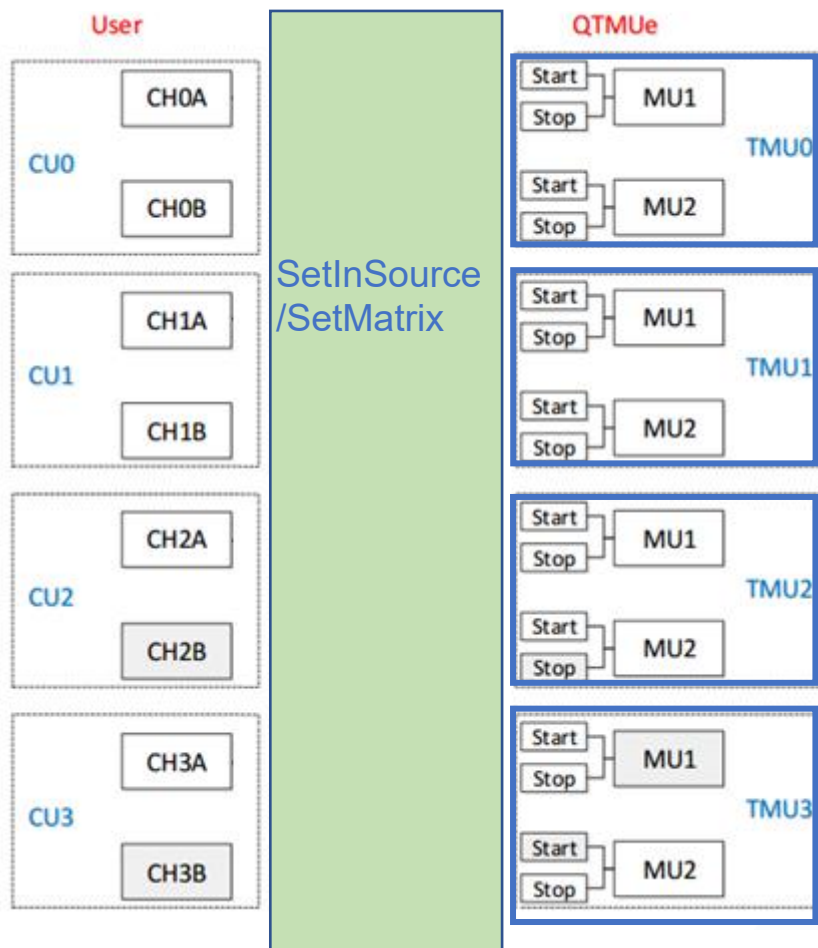
输入通道CHA 和CHB 在功能上完全一致，物理上完全独立，都可以用来设定为Start和Stop信号。

4. 多工位

QTMUe 以测量单元为最小操作单位，一块QTMUe 最多可以分配到四个工位。

输入通道/用户通道

测量单元/物理通道

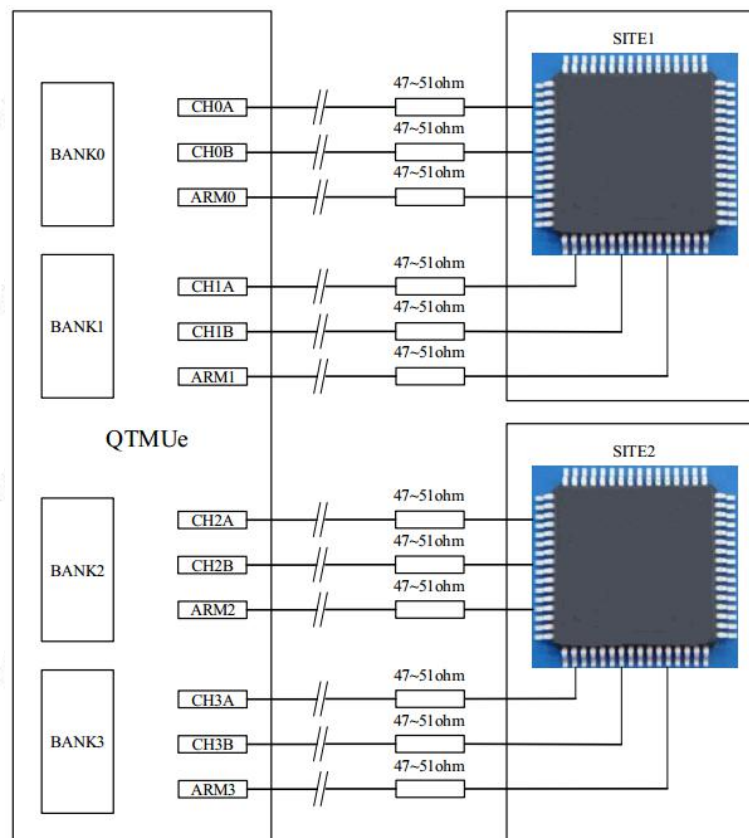


• 原理图设计、PCB布局及注意事项

5. 信号连接

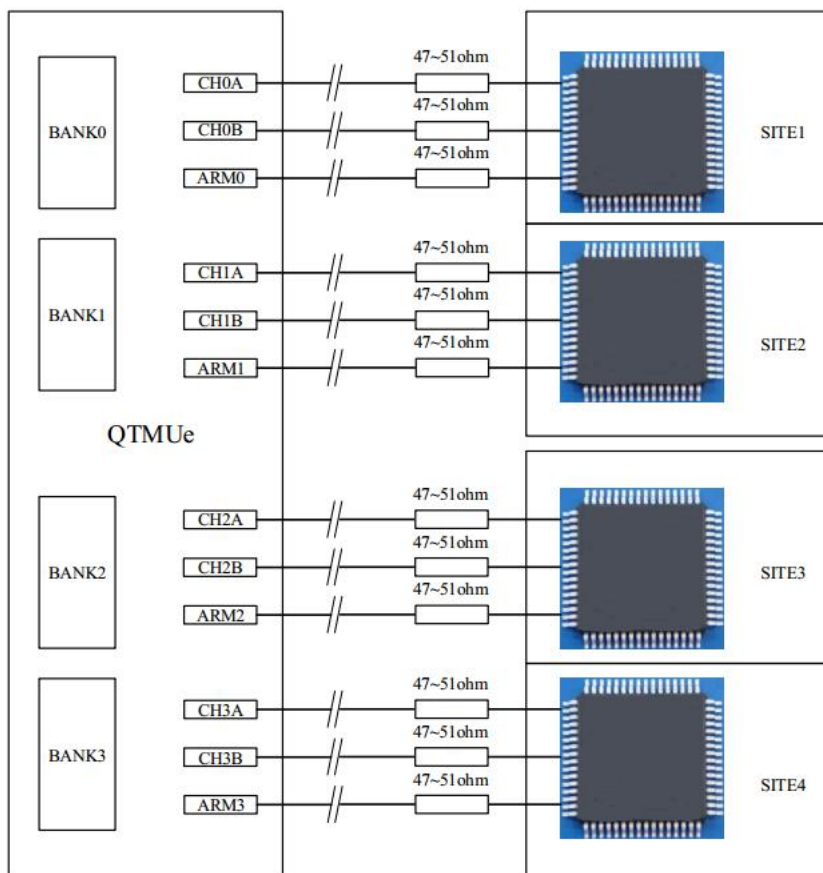
1) 考虑多工位并行测试的需求，建议将第一个物理通道（如下图中的BANK0）对应工位的被测器件输出连接到CH0A、CH0B和ARM0（第一组用户通道），第二个物理通道（如下图中的BANK1）对应工位的被测器件输出连接到CH1A、CH1B和ARM1，依次类推，如图所示。如不使用外部触发功能，ARM0~ARM3 无需连接。

建议的双工位信号连接示意图



• 原理图设计、PCB布局及注意事项

- 2) 在测量传输延迟等双信号时间时，可以将两个信号任意连接到同一BANK内对应的CHA和CHB。
- 3) 可以实现八个通道单信号时间同时测量，单工位时，只需将八个信号连接到QTMUe的CH0A~CH3B输入接口；多工位时，需遵从 1) 的要求。



建议的四工位信号连接示意图

- QTMUe

Connect ()

Disconnect ()

SetInSource ()

Start ()

Stop ()

Measure ()

GetMeasureResult ()

SetMatrix ()

SetArm ()

StopArm ()

• QTMUe

Connect (QTMUe_RELAY_CHANNEL *RelayChabSel*,
UINT *InoutDelay* = 500);

Remarks

接通QTMUe 当前逻辑通道的输入继电器。

Parameters

RelayChabSel

设置接通继电器通道:

QTMUe_RELAY_CHA 只接通CHA继电器

QTMUe_RELAY_CHB 只接通CHB继电器

QTMUe_RELAY_CHAB 只接通CHAB继电器

InoutDelay

设置接通继电器后的延时时间，单位uS，取值范围为 0~10000，默认值为500。

注意:

- 1) 默认值 500uS 为继电器接通所必需的时间;
- 2) 延时时间可以共享以提高测试效率，例如:

```
qtmue0.Connect(QTMUe_RELAY_CHA,0);  
qtmue1.Connect(QTMUe_RELAY_CHA,500);
```

- 3) 在函数 Connect()之后测量函数之前，如果有其它资源动作或者其它延时，可以适当减少 Connect()中的延时时间以提高测试效率，例如:

```
qtmue0.Connect(QTMUe_RELAY_CHA,300);
```

• QTMUe

```
Disconnect(QTMUe_RELAY_CHANNEL RelayChabSel,
UINT InoutDelay = 100);
```

Remarks

断开当前逻辑通道的输入继电器。

Parameters

RelayChabSel

设置断开继电器通道:

QTMUe_RELAY_CHA 只断开CHA继电器

QTMUe_RELAY_CHB 只断开CHB继电器

QTMUe_RELAY_CHAB 只断开CHAB继电器

InoutDelay

设置断开继电器后的延时时间，单位uS，取值范围为 0~10000，默认值为100。

注意:

- 1) 默认值 100uS 为继电器断开所必需的时间;
- 2) 延时时间可以共享以提高测试效率，例如:

```
qtmue0.Disconnect(QTMUe_RELAY_CHA,0);
qtmue1.Disconnect(QTMUe_RELAY_CHA,500);
```

- 3) 在函数 Disconnect()之后测量函数之前，如果有其它资源动作或者其它延时，可以适当减少 Disconnect()中的延时时间以提高测试效率，例如:

```
qtmue0.Connect(QTMUe_RELAY_CHA,0);
```

• QTMUe

`SetInSource(QTMUe_SOURCE_ABsourceSel = QTMUe_SINGLE_SOURCE_A)`

Remarks

- (1) 设置当前测量信号源是单信号源还是双信号源，`Start` 和 `Stop` 信号分别是 `SOURCE_A` 还是 `SOURCE_B`。
- (2) 设置用户端通道单元和板卡内部测量单元的矩阵关系，此时，用户端通道单元和板卡内部测量单元为一一对应关系。矩阵关系请参考下面 `Example`。

注意:

- 1) 若使用该函数设置测量信号源，则默认使用的时间测量单元 (TMU) 为 MU1;
- 2) 该函数必须在 `Start` 和 `Stop` 函数之前设置，保证实际设置到硬件的 TMU 和 `Start` (`Stop`) 函数中参数相关的 TMU 保持一致;
- 3) 由 2) 延伸，如果 `SetInSource` 在 `Start` (`Stop`) 函数之后设置，可能会导致两者的 TMU 不匹配，在 `Measure` 函数中会对两者进行比较，如果不一致，则不进行测量，返回异常值;
- 4) 系统每次测试结束后 (`InitAfterTestFlow` 函数) 后调用 `SetInSource(QTMUe_SINGLE_SOURCE_A)`，恢复默认单信号状态。
- 5) 如果需要设置复杂的矩阵关系，请参考函数 `SetMatrix`。

- QTMUe

SetInSource(QTMUe_SOURCE_ABsourceSel = QTMUe_SINGLE_SOURCE_A)

Parameters

sourceSel 测量信号源选择。

QTMUe_SINGLE_SOURCE_A:

单信号源测试，Start 和Stop 信号源都是SOURCE_A

QTMUe_SINGLE_SOURCE_B:

单信号源测试，Start 和Stop 信号源都是SOURCE_B

QTMUe_DUAL_SOURCE_START_A:

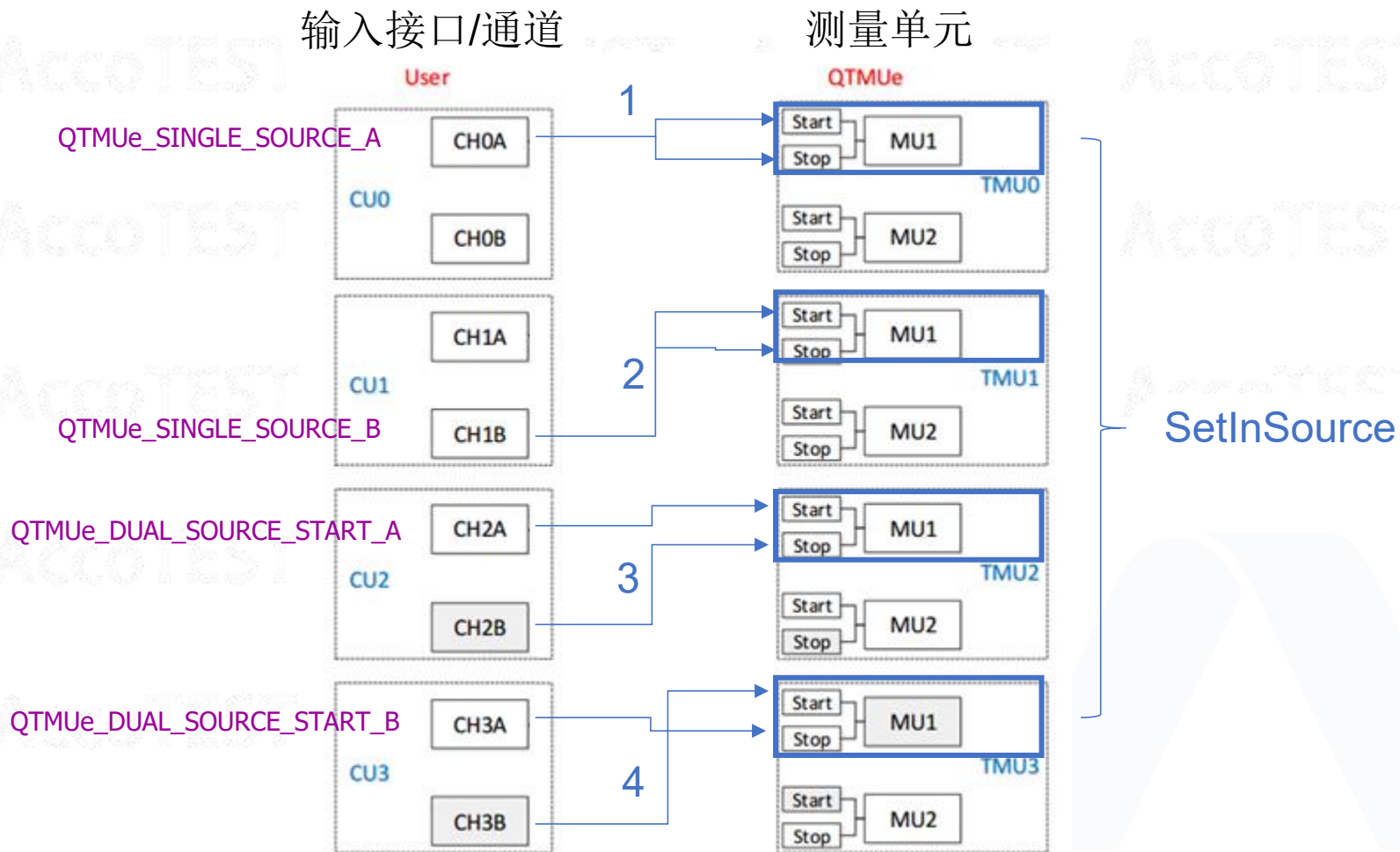
双信号源测试，Start 和Stop 信号源分别是SOURCE_A 和 SOURCE_B

QTMUe_DUAL_SOURCE_START_B:

双信号源测试，Start 和Stop 信号源分别是SOURCE_B 和 SOURCE_A.

默认值为QTMUe_SINGLE_SOURCE_A。

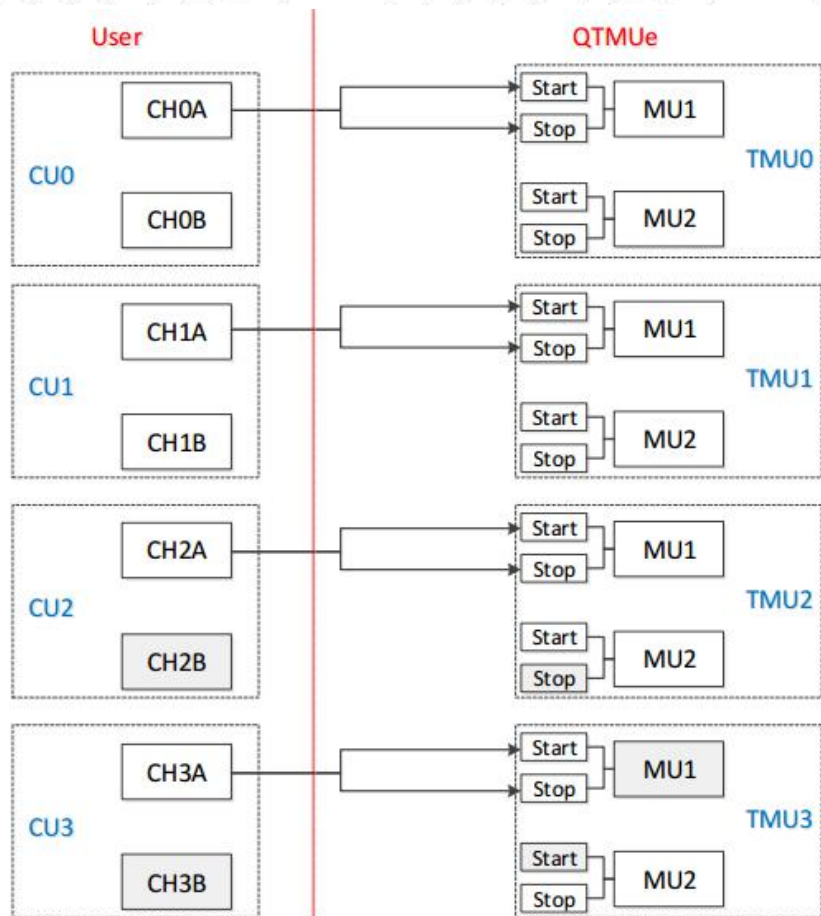
• QTMUe-SetInSource



例 1: qtmue0 的测量信号源为单信号, 且 Start 和 Stop 信号源都是SOURCE_A。

```
qtmue0.SetInSource(QTMUe_SINGLE_SOURCE_A); // Singlesource
```

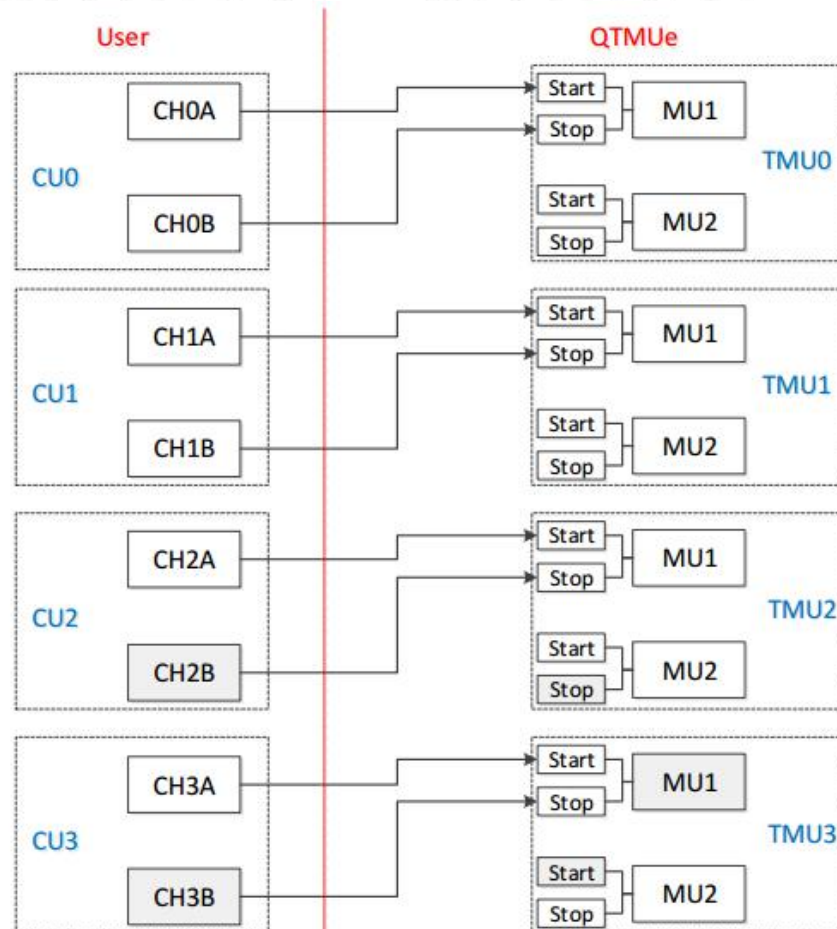
对应矩阵关系如下:



例 2: qtmue0 的测量信号源为双信号, 且 Start 和 Stop 信号源分别是SOURCE_A 和 SOURCE_B。

```
qtmue0.SetInSource(QTMUe_DUAL_SOURCE_START_A); //Dual source
```

对应矩阵关系如下:



• QTMUe

```
Start (QTMUe_TMU TmuSel = QTMUe_MU1, QTMUe_VRANGE Vrange =
QTMUe_50V,
QTMUe_SLOPE Slope = QTMUe_POS, double TriggerLevel = 0.0,
QTMUe_FILTER Filter = QTMUe_FILTER_PASS, UINT TriggerWidth =
200, UINT HoldoffTime = 0, UINT HoldoffNum = 0);
```

设置 START 信号。

注意：当进行单信号测量时，Start 和 Stop 函数的参数 VRange、Filter 也必须设置一致。

Parameters

TmuSel 设置时间测量单元

QTMUe_MU1 时间测量单元 1

QTMUe_MU2 时间测量单元 2

默认值为 QTMUe_MU1。

Vrange 设置信号电压量程。

QTMUe_50V, QTMUe_25V, QTMUe_10V, QTMUe_5V,

QTMUe_N2P6V

默认值为 QTMUe_50V。

Slope 设置比较器的触发极性

QTMUe_POS 上升沿触发

QTMUe_NEG 下降沿触发

默认值为 QTMUe_POS。

TriggerLevel 设置比较器的触发电平，单位为 V，取值范围与所选用的量程一致，默认为 0.0V。

Filter 设置滤波器，此参数在 QTMUe_N2P6V 量程下无效。

QTMUe_FILTER_PASS 直通通路，不使用滤波器

QTMUe_FILTER_10MHz 10MHz 低通滤波器

QTMUe_FILTER_1MHz 1MHz 低通滤波器

QTMUe_FILTER_100KHz 100KHz 低通滤波器

默认值为 QTMUe_FILTER_PASS。

当 Vrange 设置为 QTMUe_N2P6V 时，对应 digital 模式，此时测量带宽大于 200M，设置的 Filter 无效。

务必注意输入电压范围 (-2V~+6V)，若电平范围超过 -7V~+11V，则会对 QTMUe 造成永久性损坏！

注意：

1) 如果被测信号上有高频分量，建议使用输入滤波器滤除该高频分量，否则会影响测试结果；

2) 被测信号为方波信号时不建议使用输入滤波器。

• QTMUe

Parameters

TriggerWidth 设置触发脉宽，取值范围 0~80000，步进为 5，单位 nS。默认值为 200。

如果输入值不为5的整数倍，将会内部取整为最接近输入值的5的整数倍。例如：输入值为18，内部处理为20。

注意：

被测信号到达触发电平 **TriggerLevel** 之后，若在 **TriggerWidth** 时间之内一直高于（上升沿）或低于（下降沿）**TriggerLevel**，则满足触发条件，否则，继续等待下次触发电平。

HoldoffTime

设置延迟触发时间，取值范围 0~1e9，步进为5，单位nS，默认值为0。如果输入值不为5的整数倍，将会内部取整为最接近输入值的5的整数倍。例如：输入值为 18，内部处理为 20。

HoldoffNum

忽略的 START 事件个数，取值范围 0~65535，默认值为 0。

例1：设置qtmue0的START信号电压量程为10V，上升沿3.5V触发，使用1MHz滤波器，采用MU1测量。

```
qtmue0.Start(QTMUe_MU1, QTMUe_10V, QTMUe_POS, 3.5, QTMUe_FILTER_1MHz);
```

例 2：设置 qtmue0 的 START 信号电压量程为 5V，上升沿 1.5V 触发，不使用滤波器，触发脉宽为 10nS，延迟触发时间为 0，忽略 1 个 START 事件个数，采用 MU2 测量。

```
qtmue0.Start(QTMUe_MU2, QTMUe_5V, QTMUe_POS, 1.5, QTMUe_FILTER_PASS,10,0,1);
```

• QTMUe

```
Stop(QTMUe_TMU $TmuSel$ =QTMUe_MU1,QTMUe_VRange $Vrange$ = QTMUe_50V,  
QTMUe_SLOPE $Slope$  = QTMUe_POS,double  $TriggerLevel$  = 0.0,  
QTMUe_FILTER  $Filter$  = QTMUe_FILTER_PASS,UINT  $TriggerWidth$  = 200,UINT  
 $HoldoffTime$  = 0,UINT  $HoldoffNum$  = 0);
```

Remarks

设置 STOP 信号。

注意: 1) 当进行单信号测量时， Start 和 Stop 函数的参数 VRange、 Filter 也必须设置一致。
2) 测量信号频率时， 此函数可以不用调用。

Parameters

$TmuSel$ 设置时间测量单元，默认值为 QTMUe_MU1。

QTMUe_MU1 时间测量单元 1

QTMUe_MU2 时间测量单元 2

$Vrange$ 设置信号电压量程。

QTMUe_50V, QTMUe_25V, QTMUe_10V, QTMUe_5V,
QTMUe_N2P6V

默认值为 QTMUe_50V。

$Slope$ 设置比较器的触发极性

QTMUe_POS 上升沿触发

QTMUe_NEG 下降沿触发

默认值为 QTMUe_POS。

$TriggerLevel$ 设置比较器的触发电平，单位为 V，取值范围与所选用的量程一致，默认为 0.0V。

$Filter$ 设置滤波器， 此参数在 QTMUe_N2P6V 量程下无效。

QTMUe_FILTER_PASS 直通通路，不使用滤波器

QTMUe_FILTER_10MHz 10MHz 低通滤波器

QTMUe_FILTER_1MHz 1MHz 低通滤波器

QTMUe_FILTER_100KHz 100KHz 低通滤波器

默认值为 QTMUe_FILTER_PASS。

当Vrange设置为QTMUe_N2P6V时，对应digital模式，此时测量带宽大于200M，设置的Filter无效。

务必注意输入电压范围（-2V~+6V），若电平范围超过-7V~+11V，则会对QTMUe造成永久性损坏！

注意:

- 1) 如果被测信号上有高频分量，建议使用输入滤波器滤除该高频分量，否则会影响测试结果；
- 2) 被测信号为方波信号时不建议使用输入滤波器。

• QTMUe

Parameters

TriggerWidth 设置触发脉宽，取值范围 0~80000，步进为 5，单位 nS。默认值为 200。

如果输入值不为5的整数倍，将会内部取整为最接近输入值的5的整数倍。例如：输入值为18，内部处理为20。

注意：

被测信号到达触发电平 **TriggerLevel** 之后，若在 **TriggerWidth** 时间之内一直高于（上升沿）或低于（下降沿）**TriggerLevel**，则满足触发条件，否则，继续等待下次触发电平。

HoldoffTime

设置延迟触发时间，取值范围 0~1e9，步进为5，单位nS，默认值为0。如果输入值不为5的整数倍，将会内部取整为最接近输入值的5的整数倍。例如：输入值为 18，内部处理为 20。

HoldoffNum

忽略的 STOP 事件个数，取值范围 0~65535，默认值为 0。

例1：设置qtmue0的STOP信号电压量程为10V，上升沿3.5V触发，使用1MHz滤波器，采用MU1测量。

```
qtmue0.Stop(QTMUe_MU1, QTMUe_10V, QTMUe_POS, 3.5, QTMUe_FILTER_1MHz);
```

例2：设置 qtmue0 的 STOP 信号电压量程为 5V，上升沿 1.5V 触发，不使用滤波器，触发脉宽为 10nS，延迟触发时间为 0，忽略 1 个 START 事件个数，采用 MU2 测量。

```
qtmue0.Stop(QTMUe_MU2, QTMUe_5V, QTMUe_POS, 1.5, QTMUe_FILTER_PASS,10,0,1);
```

• QTMUe

```
Measure(QTMUe_TMUTmuSel=QTMUe_MU1,QTMUe_MEAS_TYPEMeasType =
QTMUe_MEAS_TIME,UINTSampleNum = 1,DoublemsTimeout = 1,
QTMUe_TRANGETrange = QTMUe_TRANGE_MS);
```

Remarks

启动测量

注意:

- 1) 如果Measure函数设置的MU没有在SetMatrix（或SetInSource）函数中设置，则不进行测量，返回异常值；
- 2) Measure函数设置的MU在SetMatrix（或 SetInSource）函数中设置，但该MU没有在Start或Stop函数中设置，则不进行测量，返回异常值。

Parameters

TmuSel 设置时间测量单元。

QTMUe_MU1 时间测量单元 1

QTMUe_MU2 时间测量单元 2

默认值为 QTMUe_MU1。

MeasType 设置测量类型。

QTMUe_MEAS_TIME 测量时间

QTMUe_MEAS_FREQ 测量频率

QTMUe_MEAS_HIGH_DUTY 测量高电平在一个周期之内所占的时间比率

QTMUe_MEAS_LOW_DUTY 测量低电平在一个周期之内所占的时间比率

QTMUe_MEAS_EVENT 测量事件

默认值为 QTMUe_MEAS_TIME。

• QTMUe

Parameters

SampleNum 设置采样个数，取值范围为 1~65535，默认值为 1。

当 MeasType 为 QTMUe_MEAS_TIME 类型时，SampleNum 为采样次数；

当 MeasType 为 QTMUe_MEAS_FREQ 类型时，SampleNum 为周期数；

当 MeasType 为 QTMUe_MEAS_HIGH_DUTY 或 QTMUe_MEAS_LOW_DUTY 类型时，SampleNum 为跳变沿个数；

当 MeasType 为 QTMUe_MEAS_EVENT 类型时，SampleNum 为事件个数（包含 start 和 stop）。

注意：

当 MeasType 为 QTMUe_MEAS_HIGH_DUTY 或 QTMUe_MEAS_LOW_DUTY 类型时，为保证正确测试，要求 SampleNum 不小于 3 使得采样至少一个周期。并且，Start() 和 Stop() 的 slope 设置必须一个为上升沿，另一个为下降沿，函数 start() 和 stop() 中的参数 HoldoffNum 必须设置为 0。

msTimeout 设置测量等待时间，等待 QTMUe 完成设定次数采样，取值范围 0.001~2e5，单位 mS，默认值为 1mS。

注意：

如果超范围，默认设置为 1。

Trange 设置测量时间量程。

QTMUe_TRANGE_MS mS 级量程档（5nS~40S）

QTMUe_TRANGE_US uS 级量程档（800nS~4mS）

QTMUe_TRANGE_NS nS 级量程档（5nS~950nS）

默认值为 QTMUe_TRANGE_MS。

注意：

MeasType 为 QTMUe_MEAS_HIGH_DUTY、QTMUe_MEAS_LOW_DUTY 或 QTMUe_MEAS_EVENT 时，Trange 必须为 QTMUe_TRANGE_MS。

例 1：设置 qtmue0 测量时间，使用 MU1 进行测量，采样点数为 1，超时时间为 2mS。

```
qtmue0.Measure(QTMUe_MU1, QTMUe_MEAS_TIME, 1, 2);
```

• QTMUe

```
GetMeasureResult (UINT siteNo, Int retType = AVERAGE_RESULT,
QTMUe_TMU TmuSel = QTMUe_MU1, UINT ArmNo = 0);
```

Remarks

根据 Measure()函数中的 MeasType 参数类型，获取相应的测量结果。

Parameters

siteNo 工位号，选择要读取测量结果的工位。 0 代表 Site1， 1 代表 Site2.....

retype 返回结果类型。

AVERAGE_RESULT

非负值 N， $0 \leq N \leq \text{SampleNum}-1$

默认值为 AVERAGE_RESULT。

当 Measure()函数中的 *MeasType* 为 QTMUe_MEAS_TIME 且 *retType* 为 AVERAGE_RESULT 时，返回时间平均值；当 *retType* 为非负值 N 时，返回第 N+1 次采样的测量值。单位为 uS。

当 Measure()函数中的 *MeasType* 为 QTMUe_MEAS_FREQ 类型时，*retType* 参数无效，返回频率值，单位 KHz。

当 Measure()函数中的 *MeasType* 为 QTMUe_MEAS_HIGH_DUTY 或 QTMUe_MEAS_LOW_DUTY 类型时，*retType* 参数无效，返回高低占空比，单位%。

当 Measure()函数中的 *MeasType* 为 QTMUe_MEAS_EVENT 且 *retType* 为 AVERAGE_RESULT 时，返回满足触发条件的事件个数；*retType* 为非负值 N 时，返回第 N+1 次事件的时刻，单位为 uS。

TmuSel 设置时间测量单元。

QTMUe_MU1 时间测量单元1

QTMUe_MU2 时间测量单元2

默认值为 QTMUe_MU1。

注意：必须和测量时 Start()和 Stop()函数中设置的 *TmuSel* 一致。

ArmNo 获取第 N+1 次外部触发的测量结果，取值范围为 0。当前仅支持获取第 1 次外部触发的结果。

默认值为 0，参数可不填写。

• QTMUe

例 1: qtmue0 获取 SITE_1 的时间测量结果。

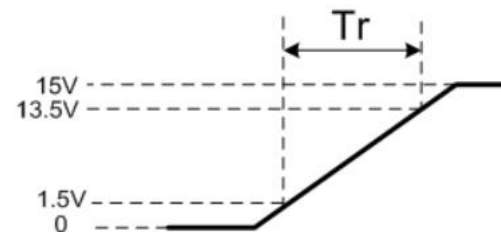
```
qtmue0.SetInSource(QTMUe_SINGLE_SOURCE_A);  
qtmue0.Start(QTMUe_MU1, QTMUe_50V, QTMUe_POS, 1.5, QTMUe_FILTER_PASS);  
//signal input to CHA, select 50V range, trigger=1.5V, rising edge  
qtmue0.Stop(QTMUe_MU1, QTMUe_50V, QTMUe_POS, 13.5, QTMUe_FILTER_PASS);  
//50V range, trigger=13.5V, rising edge  
qtmue0.Connect(QTMUe_RELAY_CHA,500); //connect CHA relay  
qtmue0.Measure(QTMUe_MU1, QTMUe_MEAS_TIME, 10, 20, QTMUe_TRANGE_MS);  
//sampleNum=10, Timeout=20mS  
qtmue0.GetMeasureResult(0, AVERAGE_RESULT,QTMUe_MU1); // Get average time of site1  
qtmue0.GetMeasureResult(0, 5, QTMUe_MU1); // Get the of sixth point time of site1
```

例 2: qtmue0 获取 SITE_1 的高占空比测量结果。

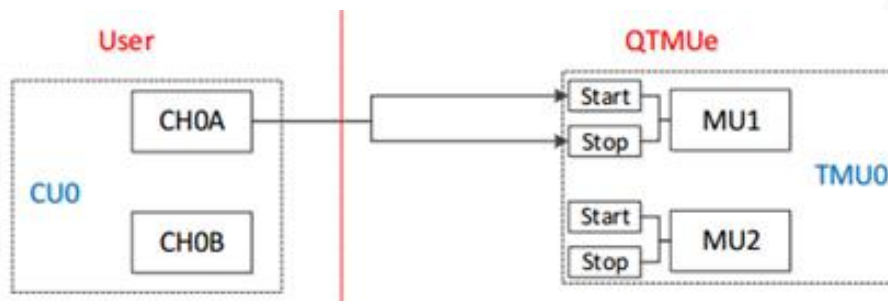
```
qtmue0.SetInSource(QTMUe_SINGLE_SOURCE_A);  
qtmue0.Start(QTMUe_MU1, QTMUe_10V, QTMUe_POS, 2.0, QTMUe_FILTER_PASS);  
qtmue0.Stop(QTMUe_MU1, QTMUe_10V, QTMUe_NEG,2.0, QTMUe_FILTER_PASS);  
qtmue0.Connect(QTMUe_RELAY_CHA,500); //connect CHA relay  
qtmue0.Measure(QTMUe_MU1, QTMUe_MEAS_HIGH_DUTY, 20, 10, QTMUe_TRANGE_MS);  
//measure high duty cycle,timeout=10mS,sample 20 edges  
qtmue0.GetMeasureResult(0, AVERAGE_RESULT, QTMUe_MU1); // Get high duty of site1.
```

• QTMUe

例 1：测试如图所示信号的上升时间 T_r 。



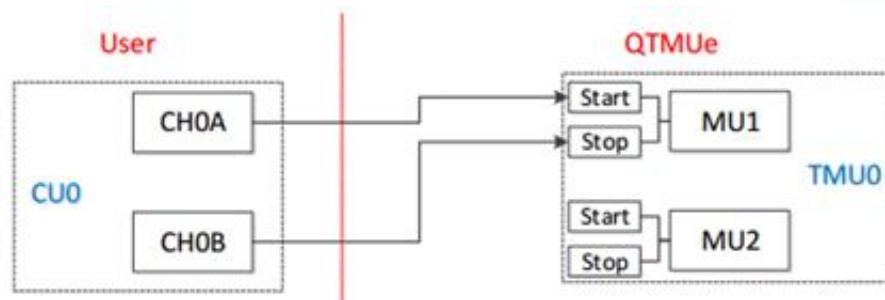
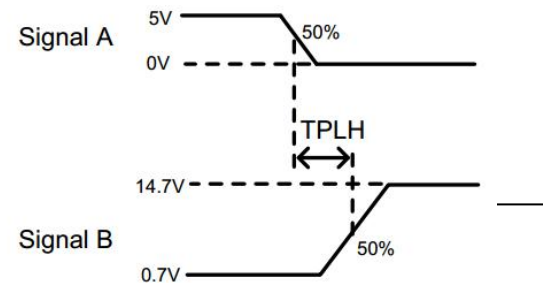
```
double val[4] = { 0.0 };
int i = 0;
qtmue0.Connect(QTMUe_RELAY_CHA,500);
qtmue0.SetInSource(QTMUe_SINGLE_SOURCE_A);
qtmue0.Start(QTMUe_MU1, QTMUe_25V, QTMUe_POS, 1.5, QTMUe_FILTER_PASS, 10);
qtmue0.Stop(QTMUe_MU1, QTMUe_25V, QTMUe_POS, 13.5, QTMUe_FILTER_PASS, 10);
qtmue0.Measure(QTMUe_MU1, QTMUe_MEAS_TIME, 1, 3, QTMUe_TRANGE_MS);
for (i = 0; i < SITENUM; i++)
{
    val[i] = qtmue0.GetMeasureResult(i, AVERAGE_RESULT, QTMUe_MU1);
    Tr->SetTestResult(i, 0, val[i]); //uS
}
qtmue0.Disconnect(QTMUe_RELAY_CHA,100); //disconnect CHA relay
```



• QTMUe

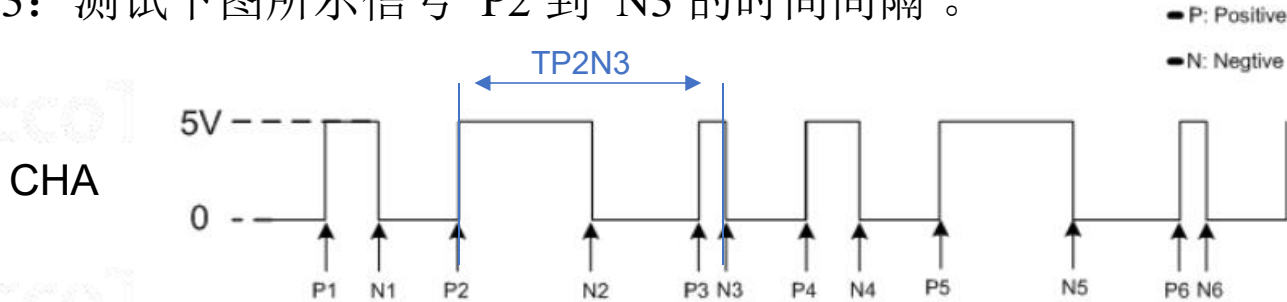
例2：测试下图所示信号的时间间隔 TPLH。

```
double val[4] = {0.0};
int i=0;
qtmue0.Connect(QTMUe_RELAY_CHAB,500);
qtmue0.SetInSource(QTMUe_DUAL_SOURCE_START_A);
qtmue0.Start(QTMUe_MU1, QTMUe_25V, QTMUe_NEG, 2.5, QTMUe_FILTER_PASS, 10);
qtmue0.Stop(QTMUe_MU1, QTMUe_25V, QTMUe_POS, 7.7, QTMUe_FILTER_PASS, 10);
qtmue0.Measure(QTMUe_MU1,QTMUe_MEAS_TIME, 10, 1 ,QTMUe_TRANGE_US);
for (i = 0; i < SITENUM; i++)
{
    val[i] = qtmue0.GetMeasureResult(i, AVERAGE_RESULT, QTMUe_MU1);
    TPLH ->SetTestResult(i, 0, val[i]); //uS
}
qtmue0.Disconnect(QTMUe_RELAY_CHAB,100); //disconnect CHA and CHB relay
```



• QTMUe

例3：测试下图所示信号 P2 到 N3 的时间间隔。



```
double val[4] = { 0.0 };
int i = 0;
qtmue0.Connect(QTMUe_RELAY_CHA,500);
qtmue0.SetInSource(QTMUe_SINGLE_SOURCE_A);
qtmue0.Start(QTMUe_MU1, QTMUe_10V, QTMUe_POS, 2.5, QTMUe_FILTER_PASS, 10, 0, 1);
// triggerWith=10ns, holdoff time=0, holdoff number=1
qtmue0.Stop(QTMUe_MU1, QTMUe_10V, QTMUe_NEG, 2.5, QTMUe_FILTER_PASS, 10, 0, 2);
// triggerWith=10ns, holdoff time=0, holdoff number=2
qtmue0.Measure(QTMUe_MU1, QTMUe_MEAS_TIME, 10, 1, QTMUe_TRANGE_US);
for (i = 0; i < SITENUM; i++)
{
    val[i] = qtmue0.GetMeasureResult(i, AVERAGE_RESULT, QTMUe_MU1);
    TP2N3->SetTestResult(i, 0, val[i]); //uS
}
qtmue0.Disconnect(QTMUe_RELAY_CHA,100);
```

• QTMUe

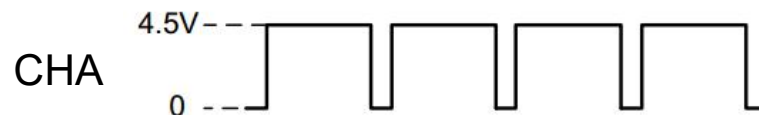
例4：测试下图所示信号的频率。



```
double val[4] = { 0.0 };
int i = 0;
qtmue0.Connect(QTMUe_RELAY_CHA,500);
qtmue0.SetInSource(QTMUe_SINGLE_SOURCE_A);
qtmue0.Start(QTMUe_MU1, QTMUe_10V, QTMUe_POS, 2.0, QTMUe_FILTER_PASS, 10);
qtmue0.Measure(QTMUe_MU1, QTMUe_MEAS_FREQ, 20,10, QTMUe_TRANGE_US);
//measure frequency, sample 20 cycles,timeout=10mS
for (i = 0; i < SITENUM; i++)
{
    val[i] = qtmue0.GetMeasureResult(i, AVERAGE_RESULT, QTMUe_MU1);
    Freq->SetTestResult(i, 0, val[i]);//KHz
}
qtmue0.Disconnect(QTMUe_RELAY_CHA,100);
```

• QTMUe

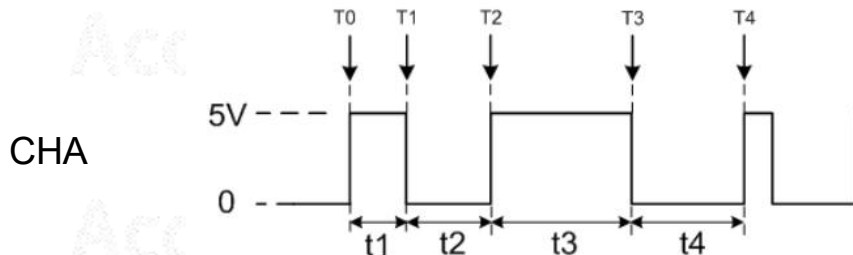
例5：测试下图所示信号的高占空比。



```
double val[4] = { 0.0 };
int i = 0;
qtmue0.Connect(QTMUe_RELAY_CHA,500);
qtmue0.SetInSource(QTMUe_SINGLE_SOURCE_A);
qtmue0.Start(QTMUe_MU1, QTMUe_10V, QTMUe_POS, 2.0, QTMUe_FILTER_PASS, 10);
qtmue0.Stop(QTMUe_MU1, QTMUe_10V, QTMUe_NEG, 2.0, QTMUe_FILTER_PASS, 10);
qtmue0.Measure(QTMUe_MU1, QTMUe_MEAS_HIGH_DUTY, 20,10,QTMUe_TRANGE_MS);
//measure high duty cycle,timeout=10mS,sample 20 edges
//Apply excitation to generate signal
for (i = 0; i < SITENUM; i++)
{
    val[i] = qtmue0.GetMeasureResult(i, AVERAGE_RESULT, QTMUe_MU1);
    Duty->SetTestResult(i, 0, val[i]);/%
}
qtmue0.Disconnect(QTMUe_RELAY_CHA,100);
```

• QTMUe

例6：例 1：测试下图所示信号，从第一个有效的上升沿开始捕捉，记录并获取五个跳变时刻（T0、T1、T2、T3、T4），并计算 t1、t2、t3、t4。



```
double Tevent[5][SITENUM] = { 0.0 };
qtmue0.Connect(QTMUe_RELAY_CHA,500);
qtmue0.SetInSource(QTMUe_SINGLE_SOURCE_A);
qtmue0.Start(QTMUe_MU1, QTMUe_10V, QTMUe_POS, 2.0, QTMUe_FILTER_PASS, 10);
qtmue0.Stop(QTMUe_MU1, QTMUe_10V, QTMUe_NEG, 2.0, QTMUe_FILTER_PASS, 10);
qtmue0.Measure(QTMUe_MU1,QTMUe_MEAS_EVENT, 5,10, QTMUe_TRANGE_MS);
//measure event, timeout=10mS, sample 5 events
//Apply excitation to generate signal
for (int SITEID = 0; SITEID < SITENUM; SITEID++){
    for (int i = 0; i < 5; i++){
        Tevent[i][SITEID] = qtmue0.GetMeasureResult(SITEID, i, QTMUe_MU1);
    }
    t1->SetTestResult(SITEID, 0, (Tevent[1][SITEID] - Tevent[0][SITEID]));
    t2->SetTestResult(SITEID, 0, (Tevent[2][SITEID] - Tevent[1][SITEID]));
    t3->SetTestResult(SITEID, 0, (Tevent[3][SITEID] - Tevent[2][SITEID]));
    t4->SetTestResult(SITEID, 0, (Tevent[4][SITEID] - Tevent[3][SITEID]));
}
qtmue0.Disconnect(QTMUe_RELAY_CHA);
```

• QTMUe

```
SetMatrix(QTMUe_TMU $TmuSel$ , QTMUe_CHANNEL $StartChSel$ ,  
QTMUe_CHANNEL $StopChSel$ , int  $siteNo$  = QTMUe_ALLSITE);
```

Remarks

设置用户端通道单元和板卡内部测量单元的矩阵关系，矩阵关系请参考下面 Example。

注意：

- 1) 该函数和 *SetInSource* 都是设置矩阵，实际硬件以最后一次设置为准；
- 2) 该函数设置矩阵后，一直有效，除非设置 *SetInSource*；
- 3) 该函数必须在 *Start* 和 *Stop* 函数之前设置，保证实际设置到硬件的 TMU和 *Start* (*Stop*) 函数中参数相关的 TMU 保持一致；
- 4) 由 3) 延伸，如果 *SetMatrix* 在 *Start* (*Stop*) 函数之后设置，可能会导致两者的 TMU 不匹配，在 *Measure* 函数中会对两者进行比较，如果不一致，则不进行测量，返回异常值；
- 5) 如果 *SetMatrix* 对不同工位设置不同的 MU，那么 *Start*、*Stop*、*Measure* 函数必须使用串行工位宏对每个工位的 MU 单独设置，否则在 *Measure* 函数中进行的判断，会不满足条件，不进行测量，返回异常值；
- 6) 仅当 *siteNo* 为 QTMUe_ALLSITE 时，该函数支持 Group 设置。

• QTMUe

```
SetMatrix(QTMUe_TMUTmuSel, QTMUe_CHANNELStartChSel,  
QTMUe_CHANNELStopChSel, int siteNo = QTMUe_ALLSITE);
```

Parameters

TmuSel 设置时间测量单元。

QTMUe_MU1 时间测量单元 1

QTMUe_MU2 时间测量单元 2

StartChSel 设置Start 信号源通道。QTMUe 每个物理通道分别有CHA 和CHB。

QTMUe_CHANNEL_A 逻辑通道包含的所有物理通道的 CHA

QTMUe_CHANNEL_B 逻辑通道包含的所有物理通道的 CHB

QTMUe_CHANNEL_0A 物理通道 0 的 CHA

QTMUe_CHANNEL_1A 物理通道 1 的 CHA

QTMUe_CHANNEL_2A 物理通道 2 的 CHA

QTMUe_CHANNEL_3A 物理通道 3 的 CHA

QTMUe_CHANNEL_0B 物理通道 0 的 CHB

QTMUe_CHANNEL_1B 物理通道 1 的 CHB

QTMUe_CHANNEL_2B 物理通道 2 的 CHB

QTMUe_CHANNEL_3B 物理通道 3 的 CHB

• QTMUe

```
SetMatrix(QTMUe_TMUTmuSel, QTMUe_CHANNELStartChSel,  
QTMUe_CHANNELStopChSel, int siteNo = QTMUe_ALLSITE);
```

Parameters

StopChSel 设置 Stop 信号源通道。QTMUe 每个物理通道分别有 CHA 和 CHB。

QTMUe_CHANNEL_A 逻辑通道包含的所有物理通道的 CHA

QTMUe_CHANNEL_B 逻辑通道包含的所有物理通道的 CHB

QTMUe_CHANNEL_0A 物理通道 0 的 CHA

QTMUe_CHANNEL_1A 物理通道 1 的 CHA

QTMUe_CHANNEL_2A 物理通道 2 的 CHA

QTMUe_CHANNEL_3A 物理通道 3 的 CHA

QTMUe_CHANNEL_0B 物理通道 0 的 CHB

QTMUe_CHANNEL_1B 物理通道 1 的 CHB

QTMUe_CHANNEL_2B 物理通道 2 的 CHB

QTMUe_CHANNEL_3B 物理通道 3 的 CHB

siteNo 工位选择，范围[-1, 250]，若超出范围，没有进行矩阵设置，函数返回异常值-1。

QTMUe_ALLSITE 即-1，所有工位

非负值N 某一个工位

默认值为 QTMUe_ALLSITE (-1) 。

注意:

- 1) 当*StartChSel* 或*StopChSel* 为QTMUe_CHANNEL_A 或QTMUe_CHANNEL_B 时，*siteNo* 必须为QTMUe_ALLSITE，可省缺；
- 2) 当*StartChSel* 为QTMUe_CHANNEL_A 或QTMUe_CHANNEL_B 时，*StopChSel* 也必须为QTMUe_CHANNEL_A 或 QTMUe_CHANNEL_B；
- 3) 当 *StartChSel* 为 QTMUe_CHANNEL_0A~QTMUe_CHANNEL_3B 时，*stopChSel* 也必须是 QTMUe_CHANNEL_0A~QTMUe_CHANNEL_3B，且*siteNo* 必须为具体工位号（非负数），不能为QTMUe_ALLSITE (-1)。

双工位配置，SITE_1 包含通道0和1，SITE_2包含通道2和3，qtmue0包含通道0和2，qtmue1包含通道1和3。

```
QTMUe qtmue0("S23_0, S23_2");  
QTMUe qtmue1("S23_1, S23_3");
```

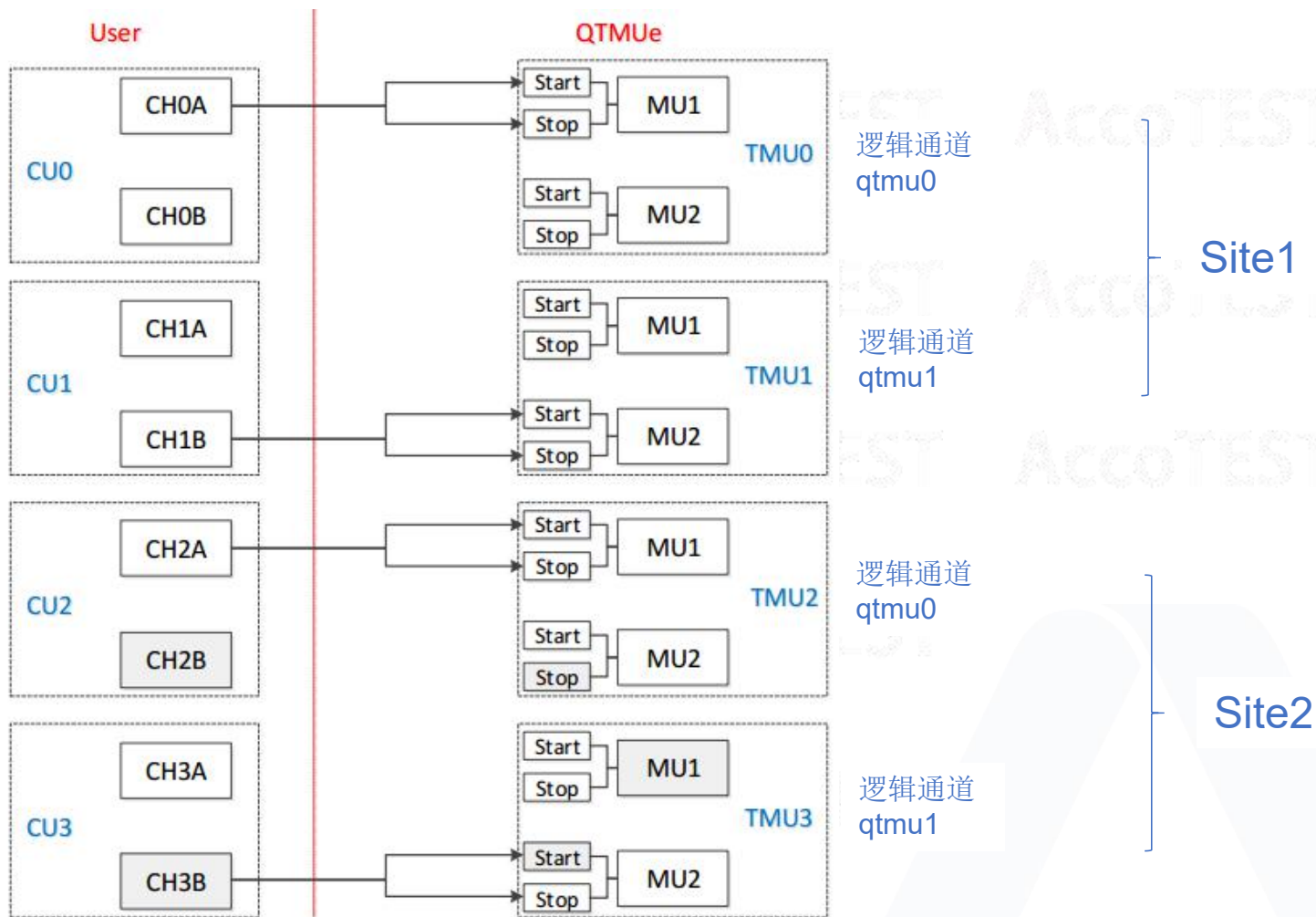
例1： qtmue0 所有工位包含的物理通道都选择测量单元为MU1，且Start和Stop 信号源都是 QTMUe_CHANNEL_A。
qtmue1 所有工位包含的物理通道都选择测量单元为MU2，且Start 和Stop信号源都是 QTMUe_CHANNEL_B。

```
qtmue0.SetMatrix(QTMUe_MU1, QTMUe_CHANNEL_A, QTMUe_CHANNEL_A, QTMUe_ALLSITE);  
qtmue1.SetMatrix(QTMUe_MU2, QTMUe_CHANNEL_B, QTMUe_CHANNEL_B, QTMUe_ALLSITE);
```

上述 qtmue0 设置的矩阵和下面 SetInSource 设置的矩阵相同

```
qtmue0.SetInSource(QTMUe_SINGLE_SOURCE_A); // Singlesource
```

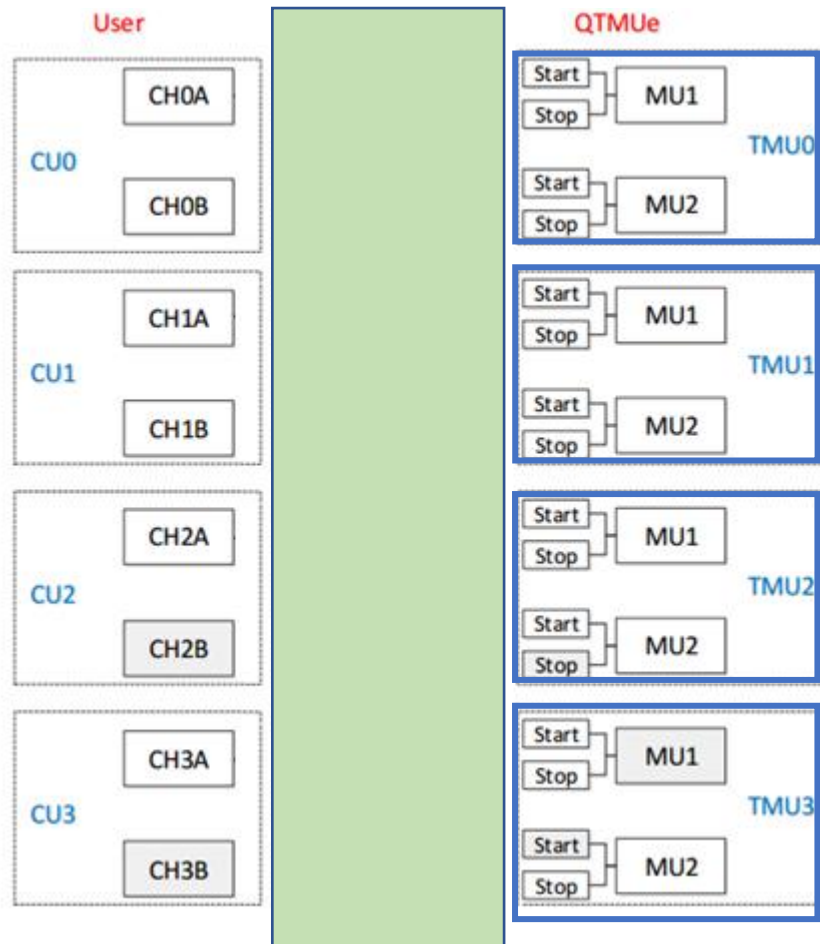
对应矩阵关系如下：



- SetMatrix()

输入接口/通道

测量单元

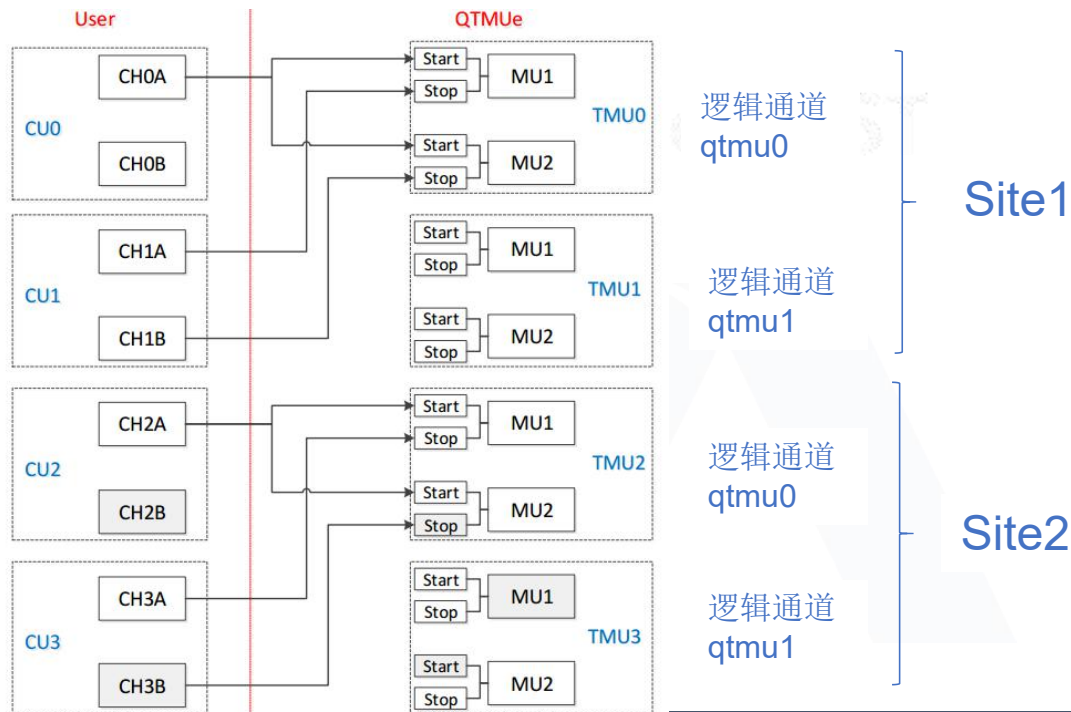


例2: qtmue0

SITE_1对应的 MU1, Start和 Stop 测量信号分别为 QTMUe_CHANNEL_0A和 QTMUe_CHANNEL_1A;
 SITE_1对应的 MU2, Start和 Stop测量信号分别为 QTMUe_CHANNEL_0A和 QTMUe_CHANNEL_1B;
 SITE_2对应的 MU1, Start和 Stop测量信号分别为 QTMUe_CHANNEL_2A和 QTMUe_CHANNEL_3A;
 SITE_2对应的 MU2, Start和 Stop测量信号分别为 QTMUe_CHANNEL_2A和 QTMUe_CHANNEL_3B;

```
qtmue0.SetMatrix(QTMUe_MU1, QTMUe_CHANNEL_0A, QTMUe_CHANNEL_1A, 0);
qtmue0.SetMatrix(QTMUe_MU2, QTMUe_CHANNEL_0A, QTMUe_CHANNEL_1B, 0);
qtmue0.SetMatrix(QTMUe_MU1, QTMUe_CHANNEL_2A, QTMUe_CHANNEL_3A, 1);
qtmue0.SetMatrix(QTMUe_MU2, QTMUe_CHANNEL_2A, QTMUe_CHANNEL_3B, 1);
```

对应矩阵关系如图:



例 3：一块板卡单工位测试，用4个物理通道的8个MU，测试8个脉冲信号脉宽。

```
QTMUe qtmue0("S23_0");  
QTMUe qtmue1("S23_1");  
QTMUe qtmue2("S23_2");  
QTMUe qtmue3("S23_3");
```

```
double Tevent[5][SITENUM] = { 0.0 };  
qtmue0.Connect(QTMUe_RELAY_CHAB);  
qtmue1.Connect(QTMUe_RELAY_CHAB);  
qtmue2.Connect(QTMUe_RELAY_CHAB);  
qtmue3.Connect(QTMUe_RELAY_CHAB);  
qtmue0.SetInSource(QTMUe_SINGLE_SOURCE_A);  
qtmue1.SetInSource(QTMUe_SINGLE_SOURCE_A);  
qtmue2.SetInSource(QTMUe_SINGLE_SOURCE_A);  
qtmue3.SetInSource(QTMUe_SINGLE_SOURCE_A);  
qtmue0.SetMatrix(QTMUe_MU2,QTMUe_CHANNEL_B,QTMUe_CHANNEL_B,QTMUe_ALLSITE);  
qtmue1.SetMatrix(QTMUe_MU2, QTMUe_CHANNEL_B, QTMUe_CHANNEL_B, QTMUe_ALLSITE);  
qtmue2.SetMatrix(QTMUe_MU2, QTMUe_CHANNEL_B, QTMUe_CHANNEL_B, QTMUe_ALLSITE);  
qtmue3.SetMatrix(QTMUe_MU2, QTMUe_CHANNEL_B, QTMUe_CHANNEL_B, QTMUe_ALLSITE);  
qtmue0.Start(QTMUe_MU1, QTMUe_10V, QTMUe_POS, 2, QTMUe_FILTER_PASS, 200);  
qtmue0.Stop(QTMUe_MU1, QTMUe_10V, QTMUe_NEG, 1, QTMUe_FILTER_PASS, 200);  
qtmue1.Start(QTMUe_MU1, QTMUe_10V, QTMUe_POS, 2, QTMUe_FILTER_PASS, 200);  
qtmue1.Stop(QTMUe_MU1, QTMUe_10V, QTMUe_NEG, 1, QTMUe_FILTER_PASS, 200);  
qtmue2.Start(QTMUe_MU1, QTMUe_10V, QTMUe_POS, 2, QTMUe_FILTER_PASS, 200);  
qtmue2.Stop(QTMUe_MU1, QTMUe_10V, QTMUe_NEG, 1, QTMUe_FILTER_PASS, 200);  
qtmue3.Start(QTMUe_MU1, QTMUe_10V, QTMUe_POS, 2, QTMUe_FILTER_PASS, 200);  
qtmue3.Stop(QTMUe_MU1, QTMUe_10V, QTMUe_NEG, 1, QTMUe_FILTER_PASS, 200);
```

```
qtmue0.Start(QTMUe_MU2, QTMUe_10V, QTMUe_POS, 2, QTMUe_FILTER_PASS, 200);  
qtmue0.Stop(QTMUe_MU2, QTMUe_10V, QTMUe_NEG, 1, QTMUe_FILTER_PASS, 200);  
qtmue1.Start(QTMUe_MU2, QTMUe_10V, QTMUe_POS, 2, QTMUe_FILTER_PASS, 200);  
qtmue1.Stop(QTMUe_MU2, QTMUe_10V, QTMUe_NEG, 1, QTMUe_FILTER_PASS, 200);  
qtmue2.Start(QTMUe_MU2, QTMUe_10V, QTMUe_POS, 2, QTMUe_FILTER_PASS, 200);  
qtmue2.Stop(QTMUe_MU2, QTMUe_10V, QTMUe_NEG, 1, QTMUe_FILTER_PASS, 200);  
qtmue3.Start(QTMUe_MU2, QTMUe_10V, QTMUe_POS, 2, QTMUe_FILTER_PASS, 200);  
qtmue3.Stop(QTMUe_MU2, QTMUe_10V, QTMUe_NEG, 1, QTMUe_FILTER_PASS, 200);
```

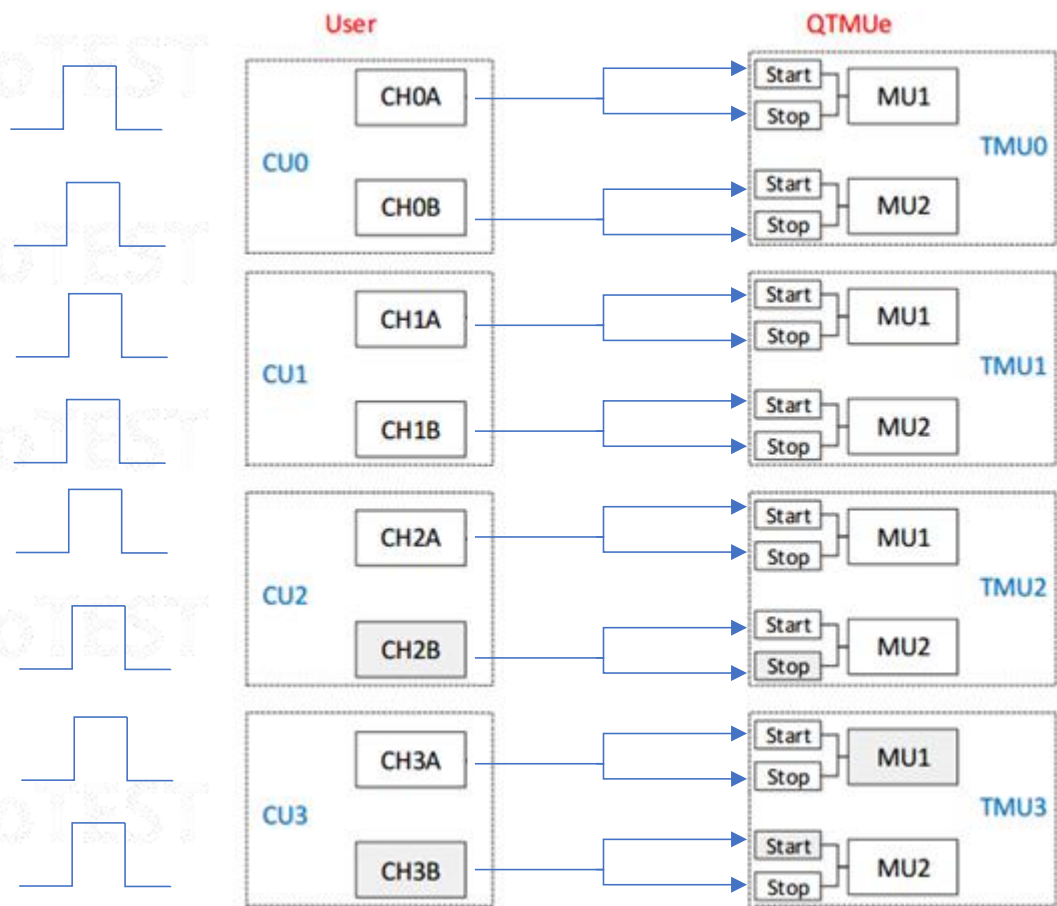
```
qtmue0.Measure(QTMUe_MU1, QTMUe_MEAS_TIME, 1, 3, QTMUe_TRANGE_MS);  
qtmue0.Measure(QTMUe_MU2, QTMUe_MEAS_TIME, 1, 3, QTMUe_TRANGE_MS);  
qtmue1.Measure(QTMUe_MU1, QTMUe_MEAS_TIME, 1, 3, QTMUe_TRANGE_MS);  
qtmue1.Measure(QTMUe_MU2, QTMUe_MEAS_TIME, 1, 3, QTMUe_TRANGE_MS);  
qtmue2.Measure(QTMUe_MU1, QTMUe_MEAS_TIME, 1, 3, QTMUe_TRANGE_MS);  
qtmue2.Measure(QTMUe_MU2, QTMUe_MEAS_TIME, 1, 3, QTMUe_TRANGE_MS);  
qtmue3.Measure(QTMUe_MU1, QTMUe_MEAS_TIME, 1, 3, QTMUe_TRANGE_MS);  
qtmue3.Measure(QTMUe_MU2, QTMUe_MEAS_TIME, 1, 3, QTMUe_TRANGE_MS);
```

```
double time1_mu1 = qtmue0.GetMeasureResult(0, AVERAGE_RESULT, QTMUe_MU1);  
double time1_mu2 = qtmue0.GetMeasureResult(0, AVERAGE_RESULT, QTMUe_MU2);  
double time2_mu1 = qtmue1.GetMeasureResult(0, AVERAGE_RESULT, QTMUe_MU1);  
double time2_mu2 = qtmue1.GetMeasureResult(0, AVERAGE_RESULT, QTMUe_MU2);  
double time3_mu1 = qtmue2.GetMeasureResult(0, AVERAGE_RESULT, QTMUe_MU1);  
double time3_mu2 = qtmue2.GetMeasureResult(0, AVERAGE_RESULT, QTMUe_MU2);  
double time4_mu1 = qtmue3.GetMeasureResult(0, AVERAGE_RESULT, QTMUe_MU1);  
double time4_mu2 = qtmue3.GetMeasureResult(0, AVERAGE_RESULT, QTMUe_MU2);
```

```
qtmue0.Disconnect(QTMUe_RELAY_CHAB);  
qtmue1.Disconnect(QTMUe_RELAY_CHAB);  
qtmue2.Disconnect(QTMUe_RELAY_CHAB);  
qtmue3.Disconnect(QTMUe_RELAY_CHAB);
```

输入接口/通道

测量单元



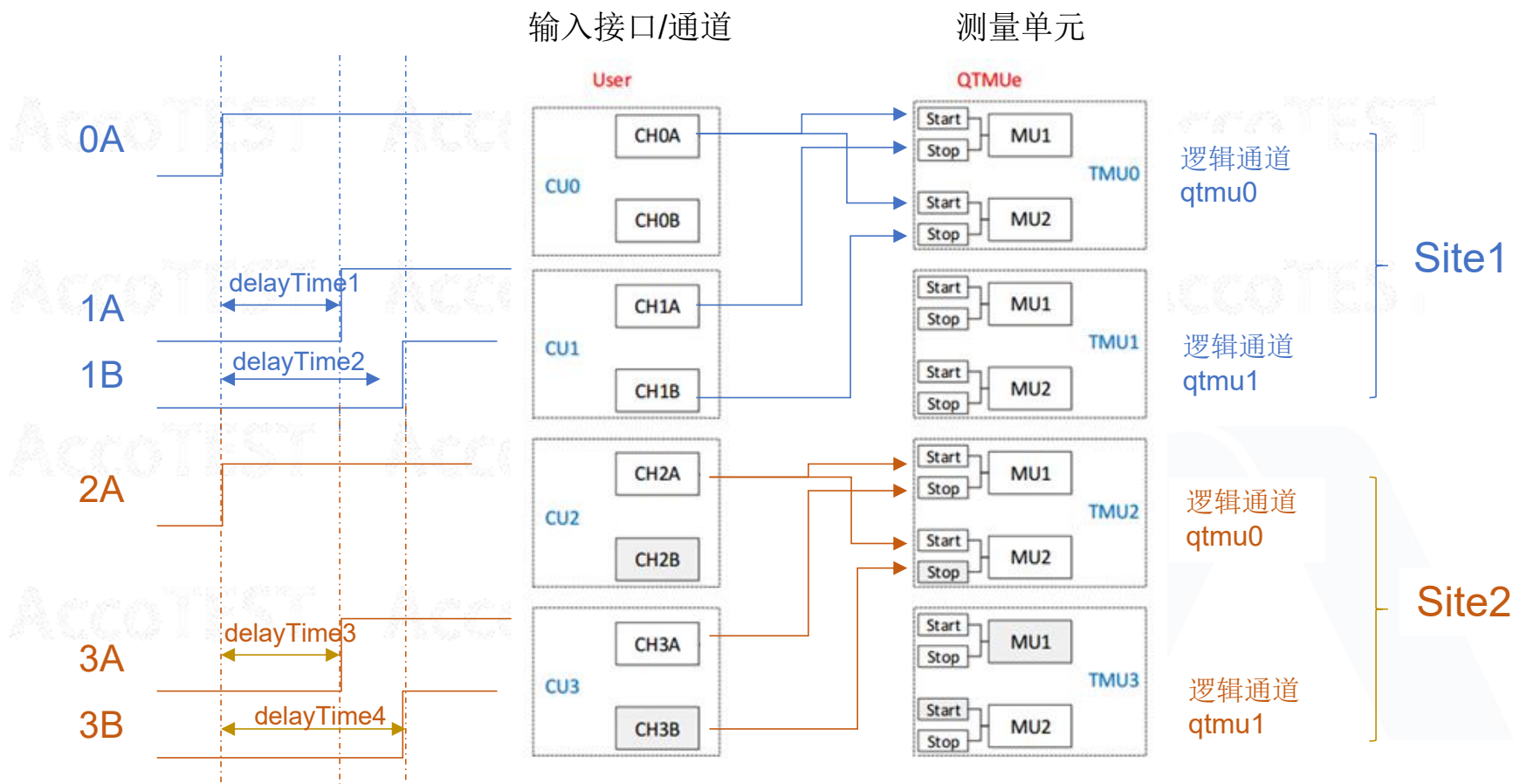
例4：一块板卡双工位设置，

SITE_1 包含物理通道0和1，SITE_2 包含物理通道2和3，qtmue0 包含物理通道0和2，qtmue1 包含物理通道1和3。

对于 SITE_1，测量 0A 到 1A 的传输延时，同时测量 0A 到 1B 的传输延时。

对于 SITE_2，测量 2A 到 3A 的传输延时，同时测量 2A 到 3B 的传输延时。

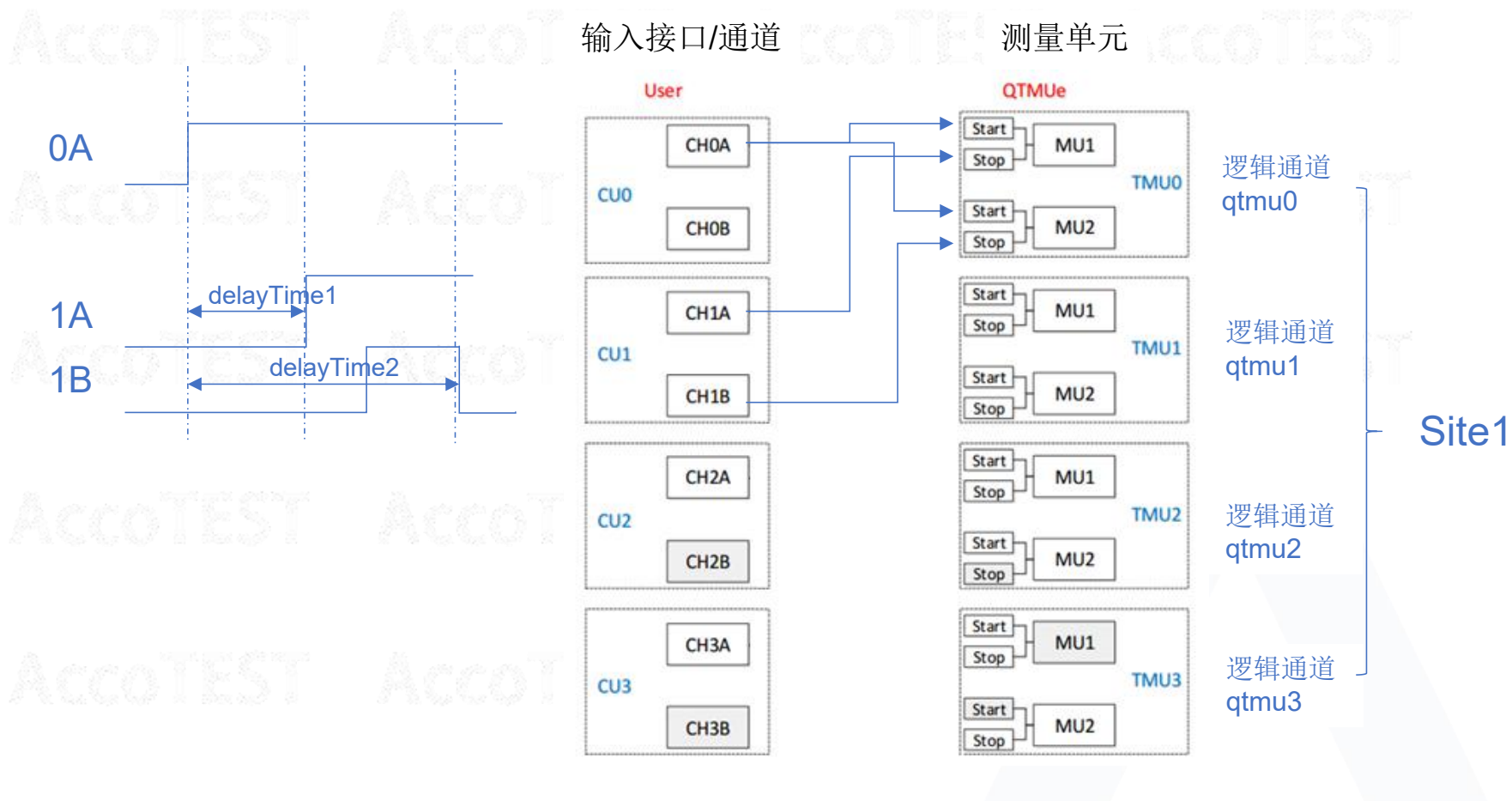
(TMU1 和 TMU2 都是上升沿到上升沿)



```
QTMUe qtmue0("S23_0, S23_2");  
QTMUe qtmue1("S23_1, S23_3");
```

```
qtmue0.Connect(QTMUe_RELAY_CHA);  
qtmue1.Connect(QTMUe_RELAY_CHAB);  
qtmue0.SetMatrix(QTMUe_MU1, QTMUe_CHANNEL_0A, QTMUe_CHANNEL_1A, 0); //Set start&stop channel to MU1 for site1  
qtmue0.SetMatrix(QTMUe_MU1, QTMUe_CHANNEL_2A, QTMUe_CHANNEL_3A, 1); //Set start&stop channel to MU1 for site2  
qtmue0.SetMatrix(QTMUe_MU2, QTMUe_CHANNEL_0A, QTMUe_CHANNEL_1B, 0); //Set start&stop channel to MU2 for site1  
qtmue0.SetMatrix(QTMUe_MU2, QTMUe_CHANNEL_2A, QTMUe_CHANNEL_3B, 1); //Set start&stop channel to MU2 for site2  
qtmue0.Start(QTMUe_MU1, QTMUe_10V, QTMUe_POS, 2, QTMUe_FILTER_PASS, 200);  
qtmue0.Stop(QTMUe_MU1, QTMUe_10V, QTMUe_POS, 2, QTMUe_FILTER_PASS, 200);  
qtmue0.Start(QTMUe_MU2, QTMUe_10V, QTMUe_POS, 2, QTMUe_FILTER_PASS, 200);  
qtmue0.Stop(QTMUe_MU2, QTMUe_10V, QTMUe_POS, 2, QTMUe_FILTER_PASS, 200);  
qtmue0.Measure(QTMUe_MU1, QTMUe_MEAS_TIME, 1, 3, QTMUe_TRANGE_MS); //MU1 in site1&site2 start measurement  
qtmue0.Measure(QTMUe_MU2, QTMUe_MEAS_TIME, 1, 3, QTMUe_TRANGE_MS); //MU2 in site1&site2 start measurement  
double delayTime1 = qtmue0.GetMeasureResult(0, AVERAGE_RESULT, QTMUe_MU1);  
double delayTime2 = qtmue0.GetMeasureResult(0, AVERAGE_RESULT, QTMUe_MU2);  
double delayTime3 = qtmue0.GetMeasureResult(1, AVERAGE_RESULT, QTMUe_MU1);  
double delayTime4 = qtmue0.GetMeasureResult(1, AVERAGE_RESULT, QTMUe_MU2);  
qtmue0.Disconnect(QTMUe_RELAY_CHAB);  
qtmue1.Disconnect(QTMUe_RELAY_CHAB);
```

例5：一块板卡单工位设置，测量 0A 到 1A 的传输延时，同时测量 0A 到 1B的传输延时。
（TMU1：上升沿到上升沿； TMU2：上升沿到下降沿）。



```
QTMUe qtmue0("S23_0");  
QTMUe qtmue1("S23_1");  
QTMUe qtmue2("S23_2");  
QTMUe qtmue3("S23_3");
```

```
qtmue0.Connect(QTMUe_RELAY_CHA);  
qtmue1.Connect(QTMUe_RELAY_CHAB);  
qtmue0.SetMatrix(QTMUe_MU1, QTMUe_CHANNEL_0A, QTMUe_CHANNEL_1A, 0);//Set start&stop to MU1 for site1  
qtmue0.SetMatrix(QTMUe_MU2, QTMUe_CHANNEL_0A, QTMUe_CHANNEL_1B, 0);//Set start&stop to MU2 for site1  
qtmue0.Start(QTMUe_MU1, QTMUe_10V, QTMUe_POS, 2, QTMUe_FILTER_PASS, 200);  
qtmue0.Stop(QTMUe_MU1, QTMUe_10V, QTMUe_POS, 2, QTMUe_FILTER_PASS, 200);  
qtmue0.Start(QTMUe_MU2, QTMUe_10V, QTMUe_POS, 2, QTMUe_FILTER_PASS, 200);  
qtmue0.Stop(QTMUe_MU2, QTMUe_10V, QTMUe_NEG, 2, QTMUe_FILTER_PASS, 200);  
qtmue0.Measure(QTMUe_MU1, QTMUe_MEAS_TIME, 1, 3, QTMUe_TRANGE_MS);  
qtmue0.Measure(QTMUe_MU2, QTMUe_MEAS_TIME, 1, 3, QTMUe_TRANGE_MS);  
double delayTime1 = qtmue0.GetMeasureResult(0, AVERAGE_RESULT, QTMUe_MU1);  
double delayTime2 = qtmue0.GetMeasureResult(0, AVERAGE_RESULT, QTMUe_MU2);  
qtmue0.Disconnect(QTMUe_RELAY_CHAB);  
qtmue1.Disconnect(QTMUe_RELAY_CHAB);
```

• QTMUe

```
SetArm(QTMUe_TMU TmuSel = QTMUe_MU1, QTMUe_TRIGGER_SOURCE Source =  
QTMUe_ARM_SOURCE, QTMUe_ARM_MODE TriggerMode = QTMUe_EDGE,  
QTMUe_SLOPE Slope = QTMUe_POS, UINT Holdoff = 0, UINT ArmNum = 1);
```

Remarks

设置外部触发条件。

注意:

- 1) 外部触发源的设置范围为: $2V \leq \text{高电平} \leq 5.5V$, $0 \leq \text{低电平} \leq 0.8V$ 。
- 2) 如果使用外部触发, 则 *SetArm* 函数的参数 *TmuSel* 必须和 *Start*、*Stop* 设置一样。
- 3) 不调用该函数时, 默认为自动触发。

Parameters

TmuSel 设置时间测量单元。

QTMUe_MU1 时间测量单元 1

QTMUe_MU2 时间测量单元 2

默认值为 QTMUe_MU1。

Source 设置外部触发源, 每个物理通道有一个触发输入。

QTMUe_ARM_SOURCE 逻辑通道包含的所有物理通道的触发输入

QTMUe_ARM_SOURCE_0 物理通道 0 的触发输入

QTMUe_ARM_SOURCE_1 物理通道 1 的触发输入

QTMUe_ARM_SOURCE_2 物理通道 2 的触发输入

QTMUe_ARM_SOURCE_3 物理通道 3 的触发输入

默认值为 QTMUe_ARM_SOURCE。

TriggerMode 设置触发模式

QTMU_EDGE 边沿触发

QTMU_LEVEL 电平触发

默认值为 QTMUe_EDGE。

• QTMUe

Parameters

<i>Slope</i>	设置比较器的触发极性/电平。 QTMUe_POS 上升沿（高电平）触发 QTMUe_NEG 下降沿（低电平）触发 默认值为 QTMUe_POS。
<i>Holdoff</i>	设置边沿计数个数/电平计数时间。默认为 0。 当为边沿触发时，取值范围为 0~65535。 当为电平触发时，取值范围 0~327675，步进为5nS，如果输入值不为 5 的整数倍，将会内部取整为最接近输入值的 5 的整数倍。例如：输入值为 18，将当作 20 处理。
<i>ArmNum</i>	设置触发次数，范围为 1，当前仅支持 1 次外部触发。默认值为 1，参数可不填写。设置外部触发条件。

例1：设置qtmue0 触发源为外部触发，边沿触发，上升沿触发，采用 MU1测量。

```
qtmue0.SetArm(QTMUe_MU1, QTMUe_ARM_SOURCE, QTMUe_EDGE, QTMUe_POS);  
// external trigger, edge trigger, rising edge
```

• QTMUe

```
StopArm(QTMUe_TMUTmuSel=QTMUe_MU1);
```

Remarks

恢复自动触发模式，且 SetArm 函数中的参数 TriggerMode、Slope、Holdoff、ArmNum 恢复默认值

Parameters

TmuSel 设置时间测量单元。

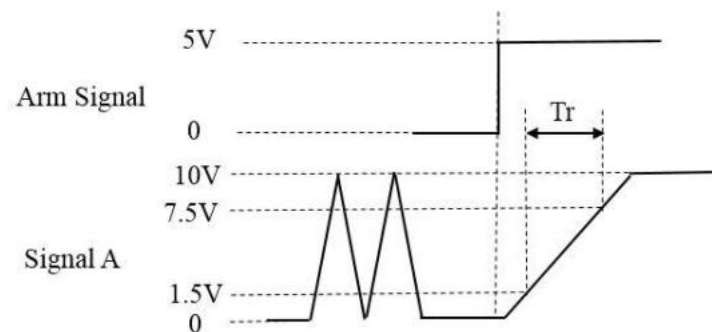
QTMUe_MU1 时间测量单元 1

QTMUe_MU2 时间测量单元 2

默认值为 QTMUe_MU1。

• QTMUe

例1：施加 0~5V 的外部 Arm 触发信号，接收到 Arm 触发信号后，测试下图所示信号的上升时间 Tr 。



```
double val[4] = { 0.0 };
int i = 0;
qtmue0.Connect(QTMUe_RELAY_CHA,500);
qtmue0.SetInSource(QTMUe_SINGLE_SOURCE_A); //CHA connected to the start&stop of MU1
qtmue0.Start(QTMUe_MU1, QTMUe_25V, QTMUe_POS, 1.5, QTMUe_FILTER_PASS, 10);
qtmue0.Stop(QTMUe_MU1, QTMUe_25V, QTMUe_POS, 7.5, QTMUe_FILTER_PASS, 10);
qtmue0.SetArm(QTMUe_MU1, QTMUe_ARM_SOURCE, QTMUe_EDGE, QTMUe_POS);
qtmue0.Measure(QTMUe_MU1, QTMUe_MEAS_TIME, 1, 100, QTMUe_TRANGE_US);
//measure time,timeout=100mS
//Apply excitation to generate external Arm signal
for (i = 0; i < SITENUM; i++)
{
    val[i] = qtmue0.GetMeasureResult(i, AVERAGE_RESULT, QTMUe_MU1);
    Tr->SetTestResult(i, 0, val[i]); //uS
}
qtmue0.Disconnect(QTMUe_RELAY_CHA,100);
```



Make Testing More Valuable